Implementation Timeline, Technology Stack & Cloud Infrastructure

This document sets out the time required for each implementation phase, the technologies proposed to build the system, and the cloud services recommended to host it — including the database. All choices are confirmed as recommendations and may be adjusted following the technical requirements discussion with the client.

1. Phase-Wise Timeline

Phase	Duration	Key activity
Phase 1 — Minimal Core Operations	**4–6 weeks**	Master data, purchase cycle, selling cycle, basic stock visibility
Phase 2 — Analytics & Forecasting	**3–4 weeks**	Ageing, excess/negative stock, projections, quote analysis, tiering
Phase 3 — Accounting (standalone)	**3–4 weeks**	Payments, expenses, assets, loans, SOA, P&L

2. Technology Stack

The system will be built as a three-tier web application — frontend, backend/API, and database — kept fully separated for easier maintenance and scaling as transaction volume grows.

Layer	Technology	Why this choice
Frontend	React.js + TypeScript	Component-based, well-suited to form-heavy screens (POs, invoices, master data); TypeScript reduces data-entry bugs through type safety
Backend / API	Node.js with Express	Same language (JavaScript/TypeScript) across frontend and backend simplifies hiring and code-sharing; large ecosystem for PDF, email, and auth libraries
Database	Amazon RDS for PostgreSQL	Relational database built for transactional integrity — critical for accounting and inventory where every number must balance; strong support for complex joins across products/orders/invoices
Caching / queue	Redis	Speeds up dashboard and report queries; queues background jobs such as PDF generation, scheduled emails, and projection recalculation
Document generation	Puppeteer / PDFKit	Generates branded PDF output for POs, invoices, delivery notes, and SOA; Puppeteer renders HTML templates to PDF for easy styling
Authentication	JWT + Role-Based Access Control	Stateless session tokens scale well; RBAC enforces the access rules confirmed in the discovery meeting (e.g. sales cannot see cost price)
File storage	S3-compatible object storage	Stores generated PDFs, uploaded supplier documents, and product images outside the database, keeping the database lean and fast

3. Cloud Infrastructure & Database Hosting

We recommend a cloud-hosted deployment over on-premise servers for this system, primarily because automated backups, patching, and disaster recovery are handled by the provider — critical given that the Accounting module will be the single source of financial truth for the business.

Component	Recommended service	Purpose
Application hosting	AWS Elastic Beanstalk or EC2 (alt: Azure App Service)	Runs the Node.js backend and serves the React frontend build
Managed database	AWS RDS for PostgreSQL (alt: Azure Database for PostgreSQL)	Automated backups, point-in-time recovery, and patching handled by the cloud provider rather than manually
Object storage	AWS S3 (alt: Azure Blob Storage)	Stores invoices, POs, delivery notes, and SOA PDFs generated by the system
CDN / static hosting	AWS CloudFront / S3 static hosting	Serves the React frontend quickly to users regardless of location
Backup & DR	Automated daily RDS snapshots, 30-day retention	Protects against data loss; point-in-time recovery for accounting and inventory data specifically
Monitoring	AWS CloudWatch (alt: Azure Monitor)	Tracks server health, error rates, and database performance; alerts the team before issues affect users
Email delivery	AWS SES (alt: SendGrid)	Sends SOA statements, quotation PDFs, and system notifications directly to customers and suppliers

Why PostgreSQL specifically for this system

- Strong transactional integrity (ACID compliance) — essential when posting a sales invoice must simultaneously deduct inventory and create a receivable without either step being lost
- Handles complex joins efficiently — required for reports that span products, customers, suppliers, and transactions together (e.g. Customer-Level Sales Report, P&L)
- Native support for numeric/decimal types with no rounding drift — important for financial figures where even small rounding errors compound over thousands of transactions
- Mature, well-supported on every major cloud provider, avoiding vendor lock-in

Environment setup recommended for all phases

- Development environment — for active build and testing of each phase's modules
- Staging environment — mirrors production for client review and sign-off before go-live of each phase
- Production environment — live system used by the business, with automated backups and monitoring active from day one